BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No: 3

Roll no:53 PRN No:12414903 Name of Student: Sara Ansingkar

Subject Name: Deep Learning

Problem Statement:

Implement forward propagation and backpropagation using TensorFlow/Keras. Analyze the effect of different learning rates and the number of epochs on model performance.

Algorithm:

1. Import the required libraries: TensorFlow, Keras, NumPy, Pandas, Matplotlib, and Scikit-learn.

2. Load the Iris dataset using Scikit-learn.

3. Split the dataset into training and testing sets.

4. Standardize the feature values using StandardScaler.

5. Create a Sequential Neural Network consisting of:

   - An input layer with 4 input features.
   - A hidden layer with 16 neurons and ReLU activation.
   - A second hidden layer with 8 neurons and ReLU activation.
   - An output layer with 3 neurons and Softmax activation.

6. Compile the model using:

   - Adam optimizer with the specified learning rate.
   - Sparse Categorical Crossentropy as the loss function.
   - Accuracy as the evaluation metric.

7. Train the model using model.fit(), where:

- **Forward Propagation** computes predictions from the input data.
- The loss is calculated by comparing predictions with the true labels.
- **Backpropagation** computes gradients of the loss with respect to the weights.
- The Adam optimizer updates the network weights.

8. Repeat the training process for different learning rates (e.g., 0.1, 0.01, 0.001) while keeping the number of epochs fixed, and record the test accuracy.

9. Repeat the training process for different numbers of epochs (e.g., 5, 10, 20) while keeping the learning rate fixed, and record the test accuracy.

10. Plot:

- Learning Rate vs Accuracy.
- Epochs vs Accuracy.
- Training and Validation Accuracy.
- Training and Validation Loss.

11. Evaluate the trained model on the test dataset and compare the results obtained for different learning rates and epoch values.

Code:

[3] ✓ 0s

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
```

[4] ✓ 0s

```
iris=load_iris()
x,y = iris.data, iris.target
x_train, x_test, y_train, y_test = train_test_split(x,y,test_size=0.2, random_state=20)
```

[6] ✓ 0s

```
def create_model(lr):
  m=Sequential([Dense(16,activation='relu',input_shape=(4,)),
                Dense(8,activation='relu'),
                Dense(3,activation='softmax')])
  m.compile(optimizer=Adam(learning_rate=lr),
            loss='sparse_categorical_crossentropy',
            metrics=['accuracy'])
  return m
```

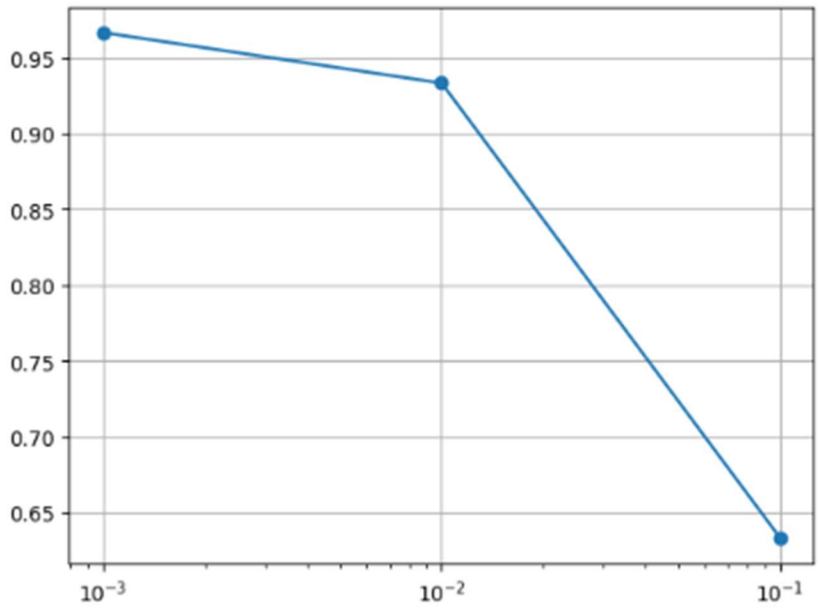
[8] ✓ 15s

```
learning_rates=[0.1,0.01,0.001]
results=[]
for lr in learning_rates:
    model=create_model(lr)
    model.fit(x_train,y_train,epochs=50,batch_size=16,verbose=0)
    loss,acc=model.evaluate(x_test,y_test,verbose=0)
    results.append([lr,acc])
df=pd.DataFrame(results,columns=['Learning Rate','Accuracy'])
print(df)
plt.plot(df['Learning Rate'],df['Accuracy'],marker='o')
plt.xscale('log'); plt.grid(); plt.show()
```


```
learning_rates=[0.1,0.01,0.001]
results=[]
for lr in learning_rates:
    model=create_model(lr)
    model.fit(x_train,y_train,epochs=50,batch_size=16,verbose=0)
    loss,acc=model.evaluate(x_test,y_test,verbose=0)
    results.append([lr,acc])
df=pd.DataFrame(results,columns=['Learning Rate','Accuracy'])
print(df)
plt.plot(df['Learning Rate'],df['Accuracy'],marker='o')
plt.xscale('log'); plt.grid(); plt.show()
```


```
WARNING:tensorflow:5 out of the last 13 calls to <function tensorflow:train
   Learning Rate  Accuracy
0          0.100  0.633333
1          0.010  0.933333
2          0.001  0.966667
```

```
2          0.001  0.966667
```

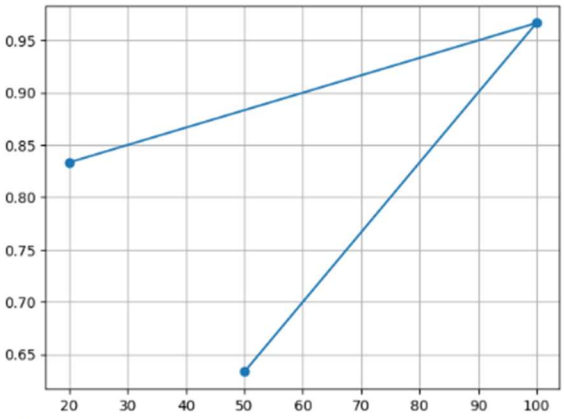


[13]

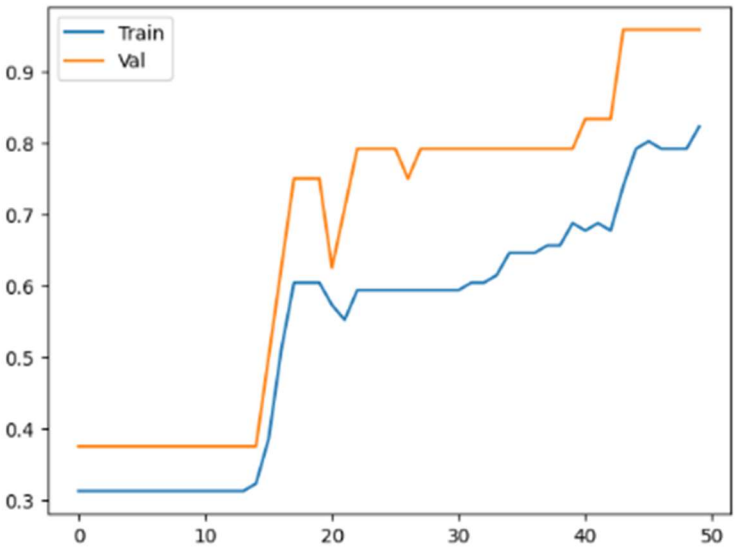
```
epochs_list=[20,50,100]
res=[]
for ep in epochs_list:
    model=create_model(0.001)
    h=model.fit(x_train,y_train,validation_split=0.2,epochs=ep,batch_size=16,verbose=0)
    loss,acc=model.evaluate(x_test,y_test,verbose=0)
    res.append([ep,acc])
edf=pd.DataFrame(res,columns=['Epochs','Accuracy'])
print(edf)
plt.plot(edf['Epochs'],edf['Accuracy'],marker='o')
plt.grid(); plt.show()

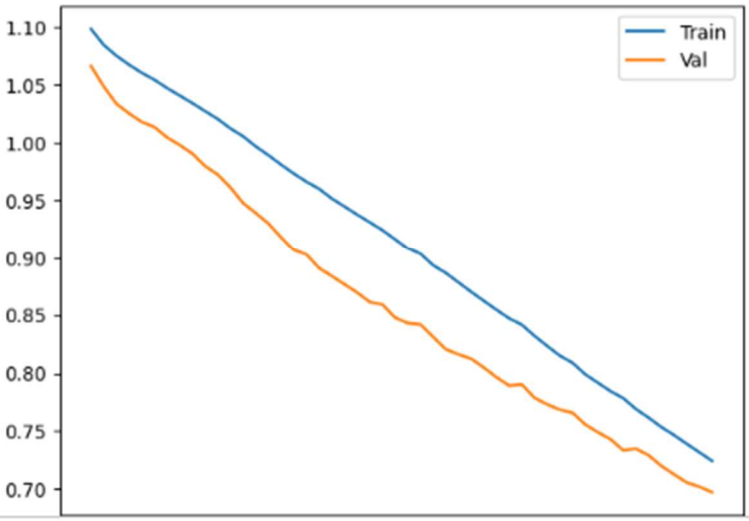
model=create_model(0.001)
history=model.fit(x_train,y_train,validation_split=0.2,epochs=50,batch_size=16)
plt.plot(history.history['accuracy']); plt.plot(history.history['val_accuracy']); plt.legend(['Train','Val']); plt.show()
plt.plot(history.history['loss']); plt.plot(history.history['val_loss']); plt.legend(['Train','Val']); plt.show()
print(model.evaluate(x_test,y_test))
```

```
   Epochs  Accuracy
0      50  0.633333
1     100  0.966667
2      20  0.833333
```




```
Epoch 1/50
```


Train
Val
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0
10
20
30
40
50


Train
Val
1.10
1.05
1.00
0.95
0.90
0.85
0.80
0.75
0.70